

Introduction to OOPS Programming

Section A: Concept Application (Theoretical – Scenario Based)

1. You are building a food delivery application. At which stage would you prefer using a compiler over an interpreter, and why? Explain based on execution speed, error detection and deployment

ANS: In a food delivery application, a compiler is preferred during the deployment and production stage because compiled programs run faster and improve application performance. A compiler also checks the whole code at once, which helps detect errors before the application is launched. This makes the software more stable and secure for users. Interpreters are useful during development and testing because they execute code line by line, making debugging easier, but compiled code is better for final deployment due to speed and reliability.

2. You are asked to design a program that calculates employee salary with multiple conditions (bonus, tax, overtime). Which programming constructs would you combine to solve this efficiently?

ANS: To design an employee salary calculation program efficiently, I would combine different programming constructs such as:

- **Variables** to store salary, bonus, tax, and overtime values.

- **Conditional statements (if-else)** to check conditions like bonus eligibility or tax deduction.
- **Loops** if salary details for multiple employees need to be processed.
- **Functions/Methods** to separate tasks like calculating bonus, overtime, and final salary for better code organization and reuse.
- **Operators** for performing mathematical calculations.

These constructs together make the program simple, organized, and efficient.

3. While developing a menu-driven console application, why are loops preferred over conditional repetition? Explain which loop you would choose and why?

ANS: Loops are preferred in a menu-driven console application because they allow the program to run continuously until the user chooses to exit. Using only conditional repetition would require writing the same code multiple times, making the program longer and harder to manage.

I would choose a while loop because it is best when the number of repetitions is not fixed. The menu can keep displaying repeatedly until the user selects the exit option. This makes the program simple, efficient, and user-friendly.

4. A student writes a program to calculate average marks using int variables and gets incorrect output. Identify the

probable cause and explain how choosing the correct data type solves the issue.

ANS: The probable cause is that the student used **int** variables, which store only whole numbers. During average calculation, decimal values are removed, leading to incorrect output.

For example, if the average is 78.5, an int variable will store it as 78 only.

Using the correct data type like **float** or **double** solves the issue because these data types can store decimal values accurately. This gives the correct average marks result.

5. You are debugging a program that compiles successfully but crashes at runtime. Which category of error is this? Mention two debugging practices you would follow to resolve it.

ANS: This is a **runtime error** because the program compiles successfully but crashes while executing.

Two debugging practices to resolve it are:

1. **Check input values and conditions** – Verify that variables, loops, arrays, or user inputs are valid and not causing issues like division by zero or invalid memory access.
2. **Use debugging tools or print statements** – Add print statements or use a debugger to trace the program flow and identify the exact line where the crash occurs.

6. You need to store student records with name, marks, and grade. Why would you choose a structure over multiple arrays in this scenario?

ANS: A structure is preferred over multiple arrays because it stores related data like student name, marks, and grade together in a single unit. This makes the program more organized, easier to manage, and reduces confusion.

Using multiple arrays can make data handling difficult because the records are stored separately, increasing the chance of errors. Structures improve readability and simplify accessing student information.

Section B: Practical Coding Tasks

1. Conditional Logic Implementation

Design a “Study Mood Assistant” that:

- Takes user input: hours studied today
- Displays motivation messages based on conditions
- Uses if-else ladder or switch case

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    int hours;
```

```
    // Taking input from user
```

```
    printf("Enter hours studied today: ");
```

```
    scanf("%d", &hours);
```

```
    // Checking conditions
```

```
    if (hours == 0) {
```

```
        printf("You should start studying today!");
```

```
}  
  
else if (hours > 0 && hours <= 2) {  
    printf("Good start! Keep studying regularly.");  
}  
  
else if (hours > 2 && hours <= 5) {  
    printf("Nice work! You are improving well.");  
}  
  
else {  
    printf("Excellent! You studied very hard today.");  
}  
  
return 0;  
}
```

2. Loops + Arrays

Create a program that:

- **Accepts 7 days of screen time data**
- **Calculates and prints:**
 - ➔ **Total screen time**
 - ➔ **Average screen time**

- **Displays a warning if average exceeds healthy limit**

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    int screenTime[7];
```

```
    int i, total = 0;
```

```
    float average;
```

```
    // Taking input for 7 days
```

```
    printf("Enter screen time for 7 days (in hours):\n");
```

```
    for(i = 0; i < 7; i++) {
```

```
        printf("Day %d: ", i + 1);
```

```
        scanf("%d", &screenTime[i]);
```

```
        total = total + screenTime[i];
```

```
    }
```

```
// Calculating average

average = total / 7.0;


// Displaying results

printf("\nTotal Screen Time = %d hours\n", total);

printf("Average Screen Time = %.2f hours\n", average);


// Checking healthy limit

if(average > 6) {

    printf("Warning: Your average screen time is too
high!\n");

}

else {

    printf("Good! Your screen time is within a healthy
limit.\n");

}


return 0;

}
```

3.Functions & Pointers

Write a program to:

- Swap two numbers using a user-defined function
- Implement swapping using pointers
- Explain why pass-by-reference is necessary here

CODE:

```
#include <stdio.h>
```

```
// User-defined function for swapping
```

```
void swap(int *a, int *b) {
```

```
    int temp;
```

```
    temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int main() {
```

```
int num1, num2;
```

```
// Taking input
```

```
printf("Enter first number: ");
```

```
scanf("%d", &num1);
```

```
printf("Enter second number: ");
```

```
scanf("%d", &num2);
```

```
// Before swapping
```

```
printf("\nBefore Swapping:\n");
```

```
printf("First Number = %d\n", num1);
```

```
printf("Second Number = %d\n", num2);
```

```
// Function call
```

```
swap(&num1, &num2);
```

```
// After swapping
```

```
printf("\nAfter Swapping:\n");
```

```
printf("First Number = %d\n", num1);  
printf("Second Number = %d\n", num2);  
  
return 0;  
}
```

4. File handling

Build a program that:

- **Writes daily goals to a file**
- **Reads and displays them on program restart**
- **Uses correct file modes**

CODE:

```
#include <stdio.h>  
  
int main() {  
    FILE *file;  
    char goal[100];  
  
    // Writing goals to file  
    file = fopen("goals.txt", "a");
```

```
if(file == NULL) {  
    printf("File cannot be opened!");  
    return 1;  
}
```

```
printf("Enter your daily goal: ");  
fgets(goal, sizeof(goal), stdin);
```

```
fprintf(file, "%s", goal);
```

```
fclose(file);
```

```
// Reading and displaying goals
```

```
file = fopen("goals.txt", "r");
```

```
if(file == NULL) {  
    printf("File cannot be opened!");  
    return 1;  
}
```

```
printf("\nSaved Daily Goals:\n");
```

```
while(fgets(goal, sizeof(goal), file) != NULL) {
```

```
    printf("%s", goal);
```

```
}
```

```
fclose(file);
```

```
return 0;
```

```
}
```

Section C: Mini Capstone Project

Mini Project: Student Productivity Tracker

Objective

Build a console-based Student Productivity Tracker that logs daily study hours and

generates a weekly report.

Your application must:

- 1. Menu-driven program**
- 2. Log daily study hours**
- 3. Generate weekly report**
- 4. Store data using files**

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file;
```

```
    int choice, i;
```

```
    float hours, total = 0, average;
```

```
    float study[7];
```

```
do {  
    printf("\n===== Student Productivity Tracker =====\n");  
    printf("1. Log Daily Study Hours\n");  
    printf("2. Generate Weekly Report\n");  
    printf("3. Exit\n");  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    switch(choice) {  
  
        case 1:  
            // Open file in append mode  
            file = fopen("studyhours.txt", "a");  
  
            if(file == NULL) {  
                printf("File cannot be opened!\n");  
                return 1;  
            }  
        }
```

```
printf("Enter study hours for today: ");
```

```
scanf("%f", &hours);
```

```
fprintf(file, "%.2f\n", hours);
```

```
fclose(file);
```

```
printf("Study hours saved successfully!\n");
```

```
break;
```

case 2:

```
// Open file in read mode
```

```
file = fopen("studyhours.txt", "r");
```

```
if(file == NULL) {
```

```
    printf("No study data found!\n");
```

```
    break;
```

```
}
```

```
total = 0;
```

```
i = 0;
```

```
// Reading data from file
```

```
while(fscanf(file, "%f", &study[i]) != EOF && i < 7) {
```

```
    total = total + study[i];
```

```
    i++;
```

```
}
```

```
fclose(file);
```

```
// Calculating average
```

```
if(i > 0) {
```

```
    average = total / i;
```

```
printf("\n===== Weekly Report =====\n");
```

```
for(int j = 0; j < i; j++) {
```

```
    printf("Day %d = %.2f hours\n", j + 1, study[j]);
```

```
}
```

```
printf("\nTotal Study Hours = %.2f\n", total);  
printf("Average Study Hours = %.2f\n", average);
```

```
if(average >= 5) {  
    printf("Excellent Productivity!\n");  
}  
else {  
    printf("Try to study more consistently.\n");  
}  
}  
else {  
    printf("No data available.\n");  
}
```

```
break;
```

case 3:

```
printf("Exiting Program...\n");  
break;
```

default:

```
printf("Invalid Choice! Please try again.\n");
```

```
}
```

```
} while(choice != 3);
```

```
return 0;
```

```
}
```